

Arealis Zord + Razorpay

Full backend implementation plan based on the actual repository structure

Prepared by Manus AI | 24 August 2026

1. Important correction after studying the actual directory

The new directory information changes the implementation recommendation. Several capabilities that were previously treated as missing already exist in Zord:

Capability	Existing location	Meaning for implementation
Webhook endpoint and webhook middleware	<code>backend/zord-edge/handler/webhook_handler.go</code> , <code>backend/zord-edge/middleware/webhook_auth.go</code>	Extend the current webhook pipeline; do not create a second webhook gateway.
Generic provider connectors	<code>backend/zord-edge/db/migrations/...create_connectors.sql</code> , <code>services/vendor_parser.go</code> , parser registry	Add Razorpay as a provider implementation in the current connector abstraction.
Bank parsing	<code>backend/zord-edge/services/bank_parser.go</code>	Extend the current parser instead of creating another bank-ingestion service.
External polling	<code>backend/zord-outcome-engine/internal/poll/</code> and <code>create_poll_schedule.sql</code>	Add Razorpay polling provider logic here; do not create a polling-only microservice for the MVP.
Settlement ingestion	<code>internal/settlement/</code> , settlement migrations, parsed rows, canonical observations	Extend existing settlement processing with Razorpay mappings.
Attachment and variance	<code>internal/attachment/</code> , attachment candidates/decisions, variance records	Reuse for Razorpay matching and under-/over-settlement.

Evidence and proof	backend/zord-evidence/ with signatures, Merkle proofs, verification, replay	Add Razorpay and bank evidence items; do not build a new proof service.
Hybrid RAG	backend/zord-prompt-layer/ with Pinecone, live SQL, hybrid retriever, citations, confidence, and guards	Extend current RAG and finance retrieval; do not create a separate RAG service.
Separate agent layer	backend/zord-agent-service/ with LangGraph agents, graphs, tools, approvals, and guardrails	Maintain agents separately from the Go prompt layer, as requested.
Scheduling	backend/zord-airflow/	Add Razorpay backfill and freshness DAGs, while keeping provider logic in Go.
Existing testing	testing/ , audit tests, functional tests, simulator, ML tests	Add provider-specific contract, integration, replay, and batch tests.

The implementation should therefore add **provider modules and bounded capabilities inside existing services**, plus a separate agent capability inside the already-existing `zord-agent-service`. A new `zord-razorpay-service` is not required for the first version.

2. Final service-boundary decision

No new polling service for the MVP

Polling should be implemented in the existing `zord-outcome-engine/internal/poll/` boundary because the outcome engine already owns `poll_schedule`, external outcome polling, settlement processing, attachment, and finality. Add a Razorpay provider client and poller under that boundary.

The responsibilities should be divided as follows:

Responsibility	Service or directory
Receive Razorpay webhooks	<code>zord-edge</code>
Authenticate and deduplicate webhooks	<code>zord-edge</code>
Store raw inbound envelopes	<code>zord-edge</code>
Call Razorpay Payments/Settlements APIs for verification and backfill	<code>zord-outcome-engine/internal/poll/</code>

Normalize Razorpay payment and settlement responses	zord-outcome-engine/internal/settlement/ or internal/poll/providers/razorpay/
Reconcile canonical intents, outcomes, settlement lines, and bank rows	zord-outcome-engine/internal/attachment/
Generate proof and Merkle evidence	zord-evidence
Build KPIs and policies	zord-intelligence
Answer finance questions using structured data and RAG	zord-prompt-layer
Plan and govern multi-step agent actions	zord-agent-service
Run periodic backfills	zord-airflow calling the outcome-engine internal endpoint or publishing a job

When a new service becomes justified

Create a separate `zord-razorpay-connector` service only if Razorpay API traffic must scale independently, polling becomes continuously high-volume, connector credentials require a separate trust boundary, or multiple consumers need a shared provider gateway. Even then, the connector should publish observations; it must not own reconciliation or proof.

3. Target repository structure after implementation

Plain Text

```
backend/
├── zord-edge/
│   ├── db/migrations/
│   │   └── 202608xx_add_provider_observation_receipts.sql
│   ├── handler/
│   │   ├── webhook_handler.go           # extend Razorpay routing
│   │   └── connector_handler.go         # connect/test/disable provid
│   ├── middleware/
│   │   └── webhook_auth.go             # preserve generic auth; call
│   ├── model/
│   │   ├── provider_observation.go     # raw observation envelope
│   │   └── razorpay_webhook.go         # provider event DTOs
│   ├── services/
│   │   ├── webhook_service.go          # existing generic flow
│   │   ├── razorpay_webhook_service.go # event validation and mappin
│   │   └── connector_service.go        # extend existing connector m
```

```

└─┬─ raw_observation_service.go          # durable raw body + hash
   │
   └─ validator/
      └─ razorpay_webhook_validator.go
   └─ testing/
      └─ razorpay_webhook_handler_test.go
      └─ razorpay_signature_test.go

└─ zord-outcome-engine/
   └─ db/migrations/
      └─ 202608xx_add_provider_observations.sql
      └─ 202608xx_add_razorpay_settlement_fields.sql
      └─ 202608xx_add_bank_observation_fields.sql
   └─ internal/poll/
      └─ poller.go                      # existing orchestration exte
      └─ schedule.go                    # existing schedule extensio
      └─ provider.go                    # generic provider interface
      └─ providers/razorpay/
         └─ client.go                   # Basic Auth HTTP client
         └─ client_config.go            # test/live base config
         └─ payments_client.go          # fetch payment/order status
         └─ settlements_client.go       # fetch settlement/recon lin
         └─ refunds_client.go           # fetch refund status if nee
         └─ pagination.go               # safe page traversal
         └─ errors.go                   # typed provider errors
         └─ poller.go                   # provider polling implement
         └─ client_test.go

   └─ internal/settlement/
      └─ razorpay_payment_mapper.go     # provider DTO → canonical o
      └─ razorpay_settlement_mapper.go  # recon DTO → canonical sett
      └─ razorpay_report_parser.go      # CSV/report input if applic
      └─ amount_units.go                # paise/minor-unit normaliza

   └─ internal/attachment/
      └─ razorpay_matcher.go            # payment → settlement line
      └─ bank_settlement_matcher.go     # UTR/amount/date matching
      └─ variance_classifier.go         # missing/under/over/duplicat
      └─ confidence.go                  # explainable score

   └─ internal/models/
      └─ provider_observation.go
      └─ razorpay_payment.go
      └─ razorpay_settlement.go
      └─ bank_observation.go
      └─ reconciliation_decision.go

   └─ internal/persistence/
      └─ provider_observation_repo.go
      └─ razorpay_payment_repo.go
      └─ razorpay_settlement_repo.go
      └─ bank_observation_repo.go
      └─ reconciliation_decision_repo.go

```

```
| | └─ internal/handler/
| |   └─ razorpay_backfill_handler.go
| |   └─ bank_statement_handler.go
| |   └─ reconciliation_handler.go
| └─ testing/
|   └─ razorpay_client_contract_test.go
|   └─ razorpay_mapper_test.go
|   └─ reconciliation_table_test.go
|   └─ polling_recovery_test.go
|   └─ batch_accuracy_test.go
|
└─ zord-evidence/
  └─ services/
    └─ razorpay_evidence.go # source-specific evidence e
    └─ bank_evidence.go
  └─ models/
    └─ observation_evidence.go
  └─ testing/
    └─ razorpay_evidence_test.go
    └─ tamper_detection_test.go
|
└─ zord-intelligence/
  └─ internal/services/
    └─ settlement_gap_intelligence_service.go
    └─ cash_forecast_service.go
    └─ reconciliation_explanation_service.go
  └─ internal/handlers/
    └─ settlement_gap_handler.go
    └─ merchant_cash_handler.go
  └─ db/migrations/
    └─ 202608xx_add_settlement_gap_projection.sql
    └─ 202608xx_add_cash_forecast_projection.sql
|
└─ zord-prompt-layer/
  └─ dto/
    └─ finance_query.go
    └─ finance_answer.go
  └─ repositories/
    └─ live_sql_razorpay.go
    └─ live_sql_bank.go
    └─ live_sql_reconciliation.go
    └─ finance_vector_repository.go
  └─ services/
    └─ finance_rag_router.go
    └─ finance_context_builder.go
    └─ finance_answer_guard.go
    └─ finance_citations.go
    └─ finance_tool_service.go
```

```
|   └─ client/
|       └─ agent_service_client.go           # optional handoff to Python
|   └─ db/
|       └─ 202608xx_create_prompt_audit.sql
|   └─ testing/
|       └─ finance_rag_test.go
|       └─ tenant_scope_test.go
|       └─ unsupported_claim_test.go
|
| └─ zord-agent-service/
|     └─ app/agents/
|         └─ settlement_investigator.py       # new agent, not a new servi
|     └─ app/graphs/
|         └─ settlement_investigation_graph.py
|     └─ app/clients/
|         └─ reconciliation_client.py
|         └─ evidence_client.py
|         └─ razorpay_connector_client.py     # read-only status/backfill
|     └─ app/tools/
|         └─ reconciliation_tools.py
|         └─ settlement_tools.py
|         └─ evidence_tools.py
|         └─ ticket_tools.py                 # draft-only initially
|     └─ app/guardrails/
|         └─ financial_limits.py
|         └─ approval_policy.py
|         └─ evidence_requirements.py
|     └─ tests/
|         └─ test_settlement_investigator.py
|         └─ test_tool_permissions.py
|         └─ test_approval_flow.py
|
| └─ zord-airflow/dags/
|     └─ razorpay_payment_backfill_dag.py
|     └─ razorpay_settlement_backfill_dag.py
|     └─ reconciliation_freshness_dag.py
|
| └─ shared/zord-event-contract/
|     └─ provider_observation_received.v1.json
|     └─ payment_observation_normalized.v1.json
|     └─ settlement_observation_normalized.v1.json
|     └─ bank_observation_normalized.v1.json
|     └─ reconciliation_completed.v1.json
|
| └─ functional-tests/razorpay/
|     └─ webhook_to_outcome.spec.ts
|     └─ backfill_recovery.spec.ts
```

```
└─ settlement_to_bank.spec.ts
└─ batch_report.spec.ts
```

4. File and function responsibilities

4.1 zord-edge : webhook and connector control

Extend the existing `handler/webhook_handler.go` rather than creating another HTTP gateway. It should route by provider and connector ID, then call `razorpay_webhook_service.go`.

Recommended functions are:

Go

```
func HandleProviderWebhook(c *gin.Context)
func (s *RazorpayWebhookService) VerifyAndPersist(ctx context.Context, raw []byte)
func (s *RazorpayWebhookService) ParseEvent(raw []byte) (RazorpayEvent, error)
func (s *RazorpayWebhookService) ToObservation(event RazorpayEvent, receipt Obs
func (s *RawObservationService) Persist(ctx context.Context, obs ProviderObserv
func (s *IdempotencyService) ReserveEvent(ctx context.Context, provider, connec
```

`webhook_auth.go` should remain the generic middleware boundary, but provider signature logic should live in `razorpay_webhook_validator.go`. The validator must use the raw request body and the connector's webhook secret. Razorpay's documentation identifies raw-body signature validation, duplicate event IDs, and event ordering as important webhook controls.[4]

The connector handler should provide:

Plain Text

```
POST /v1/connectors/razorpay/test
POST /v1/connectors/razorpay/backfill
GET /v1/connectors/razorpay/status
POST /v1/connectors/razorpay/disable
```

These routes should be authenticated by tenant/admin policy. They should never return API secrets.

4.2 zord-outcome-engine : polling and provider adapter

Implement a provider interface inside the existing polling package:

Go

```

type OutcomeProvider interface {
    FetchPayment(ctx context.Context, ref PaymentReference) (PaymentObservation
    FetchSettlementRecon(ctx context.Context, window TimeWindow, page Page) (Se
    FetchRefund(ctx context.Context, refundID string) (RefundObservation, error
    HealthCheck(ctx context.Context) error
}

```

The Razorpay client should use Basic Auth with the tenant's Test or Live key reference. It should support request timeout, retry for safe GET requests, rate-limit handling, pagination, structured redacted logs, and typed errors. The provider client must not call the reconciliation matcher.

Recommended poller functions are:

Go

```

func (p *RazorpayPoller) PollPayment(ctx context.Context, job PollJob) PollResu
func (p *RazorpayPoller) PollSettlementWindow(ctx context.Context, job PollJob)
func (p *RazorpayPoller) RepairMissingObservations(ctx context.Context, window
func (p *RazorpayPoller) PersistCursor(ctx context.Context, tenantID string, cu

```

Use the existing `poll_schedule` schema and worker orchestration. Airflow should trigger a backfill job or internal endpoint; it should not contain Razorpay business logic.

4.3 Normalizers and outcome state

`razorpay_payment_mapper.go` converts provider payment responses and webhook snapshots into the existing `canonical_outcome_events` or equivalent canonical observation model.

`razorpay_settlement_mapper.go` converts settlement-recon rows into the existing canonical settlement observations.

The mapper must preserve both provider-native and canonical values:

Go

```

type RazorpayPaymentObservation struct {
    PaymentID          string
    OrderID            string
    AmountMinor        int64
    Currency           string
    Status             string
    Captured           bool
    ErrorCode          *string
    ErrorSource        *string
    ErrorStep          *string
    ProviderCreatedAt  time.Time
}

```



```
SourceObservationID    uuid.UUID
}
```

The outcome engine must distinguish `payment_captured` , `settlement_pending` , `settled_by_psp` , `transfer_processed` , `bank_credited` , `reconciled` , and `exception` . A webhook event is an observation, not automatically finality.

4.4 Matching and variance

`razorpay_matcher.go` should first use exact stable IDs: payment ID, order ID, settlement ID, and UTR. It should then support constrained amount/currency/date matching and one-to-many allocation. `bank_settlement_matcher.go` should match bank credits by UTR first and only use amount/date fallback with reduced confidence.

Recommended function boundaries are:

Go

```
func MatchPaymentToIntent(ctx context.Context, payment PaymentObservation) MatchResult
func MatchSettlementToPayments(ctx context.Context, line SettlementLine) []MatchResult
func MatchSettlementToBank(ctx context.Context, settlement SettlementLine, bank Bank) MatchResult
func CalculateExpectedNet(input SettlementMathInput) Money
func ClassifyVariance(input VarianceInput) VarianceClassification
func CalculateConfidence(signals MatchSignals) ConfidenceResult
```

Persist the rule version, candidate IDs, signals, confidence, evidence IDs, expected net, actual net, and variance. Do not use ML to replace exact matching in the first version.

5. Schema implementation plan

The existing project already has `connectors` , `ingress_envelopes` , `canonical_settlement_observations` , `attachment_decisions` , `variance_records` , `finality_certificates` , and bank parsing infrastructure. Add only the missing columns/tables after checking the existing schema to prevent duplication.

Migration sequence

Migration	Owner	Purpose
<code>202608xx_add_provider_observations_receipts.sql</code>	<code>zord-edge</code> or outcome owner according to existing ownership	Store event ID, source type, raw hash, signature result, and ingestion status.

<code>202608xx_add_razorpay_connector_config.sql</code>	<code>zord-edge</code>	Add provider mode, secret references, health state, and backfill cursor reference to existing connectors.
<code>202608xx_add_razorpay_payment_observations.sql</code>	outcome engine	Store normalized payment snapshots and provider diagnostics.
<code>202608xx_add_razorpay_settlement_fields.sql</code>	outcome engine	Add settlement UTR, fee, tax, payment ID, order ID, line type, debit, credit, and minor-unit currency fields.
<code>202608xx_add_bank_observation_fields.sql</code>	outcome engine or edge bank-ingestion owner	Store bank account reference, source file hash, row number, posting date, UTR, amount, and direction.
<code>202608xx_add_reconciliation_evidence_refs.sql</code>	outcome engine	Link decisions and variances to evidence pack IDs and source observation IDs.
<code>202608xx_create_prompt_query_audit.sql</code>	prompt layer	Store query scope, tool calls, retrieved source IDs, answer hash, and guard result.

Before applying a migration, inspect the current table definitions and add `ALTER TABLE` statements only where the existing canonical tables cannot represent the provider fields. Do not create `razorpay_*` duplicate tables if a provider-neutral table already exists.

6. RAG and Ask Zord implementation

The actual `zord-prompt-layer` already has Pinecone, live SQL repositories, hybrid retrieval, citations, confidence, routing, and advisory guards. Therefore, do not create a new RAG service. Add a finance-specific retrieval path inside this service.

The actual `zord-agent-service` already exists separately in Python. Keep multi-step agents there. The prompt layer should answer evidence-grounded questions and expose safe read-only tools. The agent service should orchestrate investigation graphs, approvals, and action contracts.

Prompt-layer additions

`finance_rag_router.go` classifies a query as payment status, settlement status, bank proof, reconciliation, exception, cash forecast, or policy/action request. `finance_tool_service.go`

calls read-only Zord APIs for payment lifecycle, settlement lines, bank matches, decisions, exceptions, evidence packs, and batch metrics.

`finance_context_builder.go` applies tenant, merchant, time, and entity filters.

`finance_answer_guard.go` checks that every factual assertion has a structured source ID and rejects claims such as “bank credited” when there is no bank observation.

The AI should be told through a system prompt and typed answer contract:

Plain Text

You are Ask Zord Finance.

Use structured Zord tool results as financial truth. Use retrieved documents on

Never equate a payment request, authorization, or capture with bank credit.

Never equate `settlement.processed` with bank credited without a matching bank ob

Always distinguish captured, `settled_by_psp`, `transfer_processed`, `bank_credited`,

Every amount must include currency and minor/major-unit interpretation.

Every material claim must cite `payment_id`, `settlement_id`, UTR, `bank_observation`.

If sources conflict, report the conflict explicitly.

If evidence is missing, say what is missing and lower confidence.

Do not modify payment state, issue refunds, post accounting entries, or submit

Propose an action contract when operator approval is required.

The typed answer should contain `answer`, `status`, `confidence`, `facts[]`, `missing_evidence[]`, `source_ids[]`, and optional `recommended_action`. The LLM must not receive unrestricted database access or secrets.

Agent-service additions

Add `settlement_investigator.py` and `settlement_investigation_graph.py` to the existing agent service. The graph should call read-only reconciliation and evidence tools, summarize findings, ask for approval where necessary, and create a draft action contract. It should not directly call Razorpay refund or payment mutation APIs in the MVP.

7. Detailed implementation phases

Phase 0 — Repository and schema verification

Files: existing migrations, provider parser interfaces, outcome `internal/poll/`, edge webhook files, prompt-layer repositories, agent tools, and `docs/razorpay/implementation-matrix.md`.

Work: run the stack, inspect actual table definitions, identify existing provider interfaces, confirm Kafka event names, and mark each planned addition as reuse, extension, or new

file.

Testing: existing Go unit/integration tests, Compose startup, migration dry run, and contract inventory.

Output: approved architecture decision record, no-duplication schema map, and verified extension points.

Phase 1 — Razorpay connector configuration and client

Files: `zord-edge/handler/connector_handler.go` , connector service/model changes, `outcome-engine/internal/poll/providers/razorpay/*.go` , and client tests.

Work: implement Test/Live mode configuration, secret references, connection test, Basic Auth client, pagination, retry, timeout, and redacted logging.

Testing: `httptest.Server` for auth, errors, pagination, timeout, retries, rate limits, and secret leakage.

Output: successful Test Mode connection check and provider client package.

Phase 2 — Razorpay webhook observation

Files: existing edge webhook handler and middleware plus `razorpay_webhook_service.go` , validator, raw-observation model, migration, and tests.

Work: preserve raw body, verify signature, reserve event ID, persist raw body/hash, publish observation event, and handle out-of-order events.

Testing: valid/invalid signature, duplicate event, malformed payload, retry after persistence failure, event ordering, and tenant isolation.

Output: exactly-once financial observation receipt and metrics for accepted, rejected, duplicate, and failed events.

Phase 3 — Polling and API backfill

Files: existing outcome `internal/poll/` , Razorpay provider files, cursor repository, Airflow DAGs, and polling tests.

Work: poll payment status and settlement-recon data for configurable windows; repair missed webhooks; persist cursors; make overlapping windows safe; publish normalized observations.

Testing: missed webhook recovery, repeated backfill, cursor restart, pagination, API outage, expired credential, rate limit, and schema mismatch.

Output: backfill job summary with fetched, inserted, updated, duplicate, missing-webhook, failed, and cursor counts.

Phase 4 — Settlement and bank ingestion

Files: Razorpay settlement mapper, report parser, existing bank parser extension, upload handler, migrations, and CSV fixtures.

Work: normalize payments, refunds, transfers, adjustments, fee, tax, debit, credit, UTR, settlement ID, bank row, and source metadata.

Testing: valid file, duplicate file, duplicate row, missing column, wrong currency, wrong units, malformed timestamp, invalid UTR, and partial row errors.

Output: normalized settlement and bank observations with source hashes and row-level import results.

Phase 5 — Reconciliation and finality

Files: existing attachment package plus Razorpay matcher, bank matcher, state machine, amount math, variance classifier, repositories, and table tests.

Work: implement order → payment → settlement line → bank credit graph, exact-ID matching, constrained fallback, one-to-many allocation, under-/over-settlement, partial settlement, refunds, transfers, adjustments, and contradiction states.

Testing: 50–100 labeled records with ground truth, precision, recall, amount-weighted accuracy, arithmetic accuracy, proof completeness, and complete exception list.

Output: record-level decisions, batch summary, variance records, exception records, and evidence requests.

Phase 6 — Evidence and proof

Files: evidence source enrichers, observation evidence model, proof tests, and existing pack/verifier paths.

Work: include raw body hash, API response hash, report file hash, event ID, source time, receipt time, UTR, bank row, decision rule, confidence signals, and reviewer action in Merkle-rooted evidence.

Testing: deterministic proof root, tamper detection, selective disclosure, missing evidence, replay, and verification failure.

Output: independently verifiable payment-to-bank evidence pack.

Phase 7 — Ask Zord RAG and agent investigation

Files: prompt-layer finance DTOs, repositories, router, context builder, guard, tools, audit migration, agent investigator, graph, clients, and guardrail tests.

Work: structured tools first; hybrid Pinecone/live-SQL retrieval second; citations and answer guard third; agent workflow and approval contract fourth.

Testing: tenant isolation, wrong merchant, prompt injection in report text, unsupported bank-credit claim, contradictory observations, empty evidence, tool failure, and approval enforcement.

Output: cited Ask Zord response and optional signed investigation action contract.

Phase 8 — Forecasting and production hardening

Files: intelligence projections, Airflow freshness DAG, monitoring dashboards, deployment secrets, and runbooks.

Work: add settlement-lag and net-cash forecasts only after reconciliation passes. Add webhook freshness, polling lag, unmatched-value, evidence-completeness, and Ask Zord guard metrics.

Testing: load, replay, Kafka lag, database restart, Airflow retry, secret rotation, tenant isolation, and disaster recovery.

Output: production-readiness report and forecast-versus-actual report.

8. Frontend changes, deferred until backend contracts stabilize

The focus should remain backend-first. The existing `backend/zord-console` should not be redesigned now. After API contracts are stable, add or extend these pages:

Page	Backend endpoints it consumes
<code>/connectors/razorpay</code>	Connector status, connection test, last webhook, last backfill, freshness.
<code>/reconciliation/settlements</code>	Batch metrics, captured/settled/bank-credited totals, fee, tax, and net cash.
<code>/reconciliation/exceptions</code>	Variance and exception list, confidence, age, value, and action status.
<code>/transactions/:id</code>	Lifecycle timeline, observations, matching decision, and evidence references.
<code>/evidence/:packId</code>	Evidence items, hashes, Merkle proof, and verification result.
Ask Zord finance panel	Cited answers, missing evidence, confidence, and approval actions.

Polling does not require a separate page. Show its health through the Razorpay connector page and freshness metrics.

9. First five pull requests

PR 1 — Provider contract and configuration. Extend the existing connector model and add Razorpay Test/Live configuration, typed client interfaces, and contract tests.

PR 2 — Webhook observation. Extend the current edge webhook handler with Razorpay signature verification, raw-body persistence, event-ID idempotency, and observation events.

PR 3 — Polling and settlement backfill. Add the Razorpay poller under outcome-engine `internal/poll`, cursor handling, settlement-recon fetch, and Airflow DAG trigger.

PR 4 — Reconciliation and evidence. Add Razorpay settlement mapping, bank matching, variance classification, batch metrics, and proof enrichment.

PR 5 — Ask Zord and agent investigation. Add finance tools and RAG guards in the prompt layer, then add the settlement investigator graph in the existing agent service.

10. Final architecture answer

You do **not** need a new polling service now. You already have polling boundaries in `zord-outcome-engine/internal/poll/`, scheduling in `zord-airflow`, webhook infrastructure in `zord-edge`, RAG in `zord-prompt-layer`, and a separate agent service in `zord-agent-service`.

You should add a **Razorpay provider adapter**, not a duplicate platform. You should add **Razorpay-specific observation, mapping, polling, reconciliation, evidence, and finance-tool files** to the existing services. Maintain agents separately in `zord-agent-service`, while Ask Zord's structured retrieval, RAG, citations, and answer guard remain in `zord-prompt-layer`.

The first complete technical milestone is:

Plain Text

Razorpay Test Mode webhook/API

↓

existing zord-edge

↓

provider observation event

↓

existing outcome-engine poll/settlement/attachment

↓

```
bank statement observation
    ↓
variance + finality
    ↓
existing evidence service
    ↓
existing intelligence + prompt layer
    ↓
existing agent service for governed investigation
```

That design preserves the work already completed in Zord and adds only the provider-specific capabilities required to demonstrate merchant settlement truth.

References

- [1]: <https://razorpay.com/docs/webhooks/payments/?preferred-country=US> “Razorpay Docs — Payments Webhook Events”
- [2]: <https://razorpay.com/docs/webhooks/settlements/?preferred-country=US> “Razorpay Docs — Settlements Webhook Events”
- [3]: <https://razorpay.com/docs/api/settlements/fetch-recon/?preferred-country=US> “Razorpay Docs — Fetch Settlement Recon Details”
- [4]: <https://razorpay.com/docs/webhooks/validate-test/?preferred-country=US> “Razorpay Docs — Validate and Test Webhooks”
- [5]: <https://razorpay.com/docs/payments/dashboard/test-live-modes/?preferred-country=US> “Razorpay Docs — Test and Live Modes”